

APUNTES COMPLETOS DEL CURSO DE PROGRAMACIÓN EN LENGUAJE C

Fundamentos, control, funciones, archivos y estructuras de datos

Contenido

1. Fundamentos del lenguaje C
2. Entrada, salida y ficheros
3. Operadores relacionales, lógicos y condiciones
4. switch y decisiones
5. Ficheros, fechas y temperaturas
6. Bucles while y do-while
7. Procesamiento de datos en ficheros
8. Bucles for

9. Funciones y trazas

10. Vectores numéricos

11. Vectores dinámicos y cálculo de NIF

12. Cadenas y string.h

13. Matrices

14. Estructuras y typedef

15. Buenas prácticas

16. Conclusión

1. Fundamentos del lenguaje C

Lecciones 1–4: variables, operadores aritméticos, salida estándar y entrada estándar.

Variables y tipos de datos

Una variable es un espacio de memoria identificado por un nombre que almacena un valor. En C se declara indicando el tipo de dato. Los tipos básicos tratados son int para enteros, float para números reales y char para caracteres.

```
int edad = 18; float promedio = 8.5f; char letra = 'A';
```

Operadores aritméticos

Los operadores +, -, *, / y % permiten realizar operaciones aritméticas. La precedencia determina qué operaciones se realizan primero. Los paréntesis permiten modificar ese orden.

```
resultado = (8 - 3 * 2) + 7;
```

Salida estándar

printf(), de la biblioteca stdio.h, permite mostrar información en pantalla. Los especificadores de formato indican el tipo de dato.

```
#include <stdio.h> int main() { printf("Hola mundo\n"); return 0; }
```

Entrada estándar

scanf() permite recibir datos desde el teclado. En variables básicas se utiliza & para proporcionar la dirección donde se almacenará el dato.

```
int edad; printf("Edad: "); scanf("%d", &edad);
```

2. Gestión de entrada/salida y ficheros

Lecciones 5–7: cálculo de precios, archivos y procedimiento de trabajo con ficheros.

Formato de reales

Para mostrar un número real con dos decimales se puede utilizar %.2f.

```
float precio = 25.5;printf("Precio: %.2f\n", precio);
```

Ficheros

Los ficheros permiten guardar información de forma persistente. En C se utiliza FILE* junto con funciones de stdio.h. Los modos comunes son r para lectura, w para escritura y a para agregar datos.

```
FILE *archivo;archivo = fopen("datos.txt", "r");
```

Cierre y flujo de trabajo

Después de utilizar un fichero se debe cerrarlo con fclose(). Un flujo habitual es abrir, comprobar, leer o escribir, procesar y cerrar.

```
if (archivo != NULL) { /* procesar datos */ fclose(archivo);}
```

3. Operadores relacionales, lógicos y condiciones

Lecciones 8–12: comparaciones, lógica, if/else y problemas matemáticos.

Operadores relacionales

Las comparaciones incluyen <, >, <=, >=, == y !=. Permiten construir condiciones.

```
if (edad >= 18) { printf("Mayor de edad");}
```

Operadores lógicos

&& representa AND, || representa OR y ! representa NOT. Se utilizan para combinar o invertir condiciones.

```
if (edad >= 18 && edad <= 60) { printf("Rango valido");}
```

if...else

if ejecuta un bloque cuando una condición es verdadera; else permite ejecutar una alternativa.

```
if (calificacion >= 6) { printf("Aprobado");} else { printf("No aprobado");}
```

Ecuación de segundo grado

Para $ax^2 + bx + c = 0$ se utiliza el discriminante $D = b^2 - 4ac$. Si $D > 0$ hay dos

soluciones reales; si $D=0$ existe una solución real doble; si $D<0$ no existen soluciones reales.

4. Sentencia switch y ejemplos de decisión

Lecciones 13–15: selección múltiple, errores comunes y caracteres.

switch

switch permite elegir una acción según el valor de una expresión. case define alternativas, break evita continuar hacia otros casos y default maneja opciones no contempladas.

```
switch(opcion) { case 1: printf("Alta"); break; case 2: printf("Baja"); break; default: printf("Invalida"); }
```

Errores frecuentes

Es fundamental distinguir $=$, que asigna, de $==$, que compara. También se deben utilizar correctamente las llaves para delimitar bloques.

Caracteres

switch puede utilizarse para clasificar caracteres, por ejemplo identificar si una letra es una vocal.

5. Uso de ficheros para fechas y temperaturas

Lecciones 16–17: almacenamiento de temperaturas y cálculo de días.

Temperaturas

Un programa puede leer temperaturas desde un fichero, almacenarlas o procesarlas para obtener resultados.

Fechas

Para calcular días entre fechas se consideran día, mes y año, incluyendo las reglas de duración de los meses y los años bisiestos.

6. Bucles while y do-while

Lecciones 18–20: while, do-while y sumatorios.

while

Evalúa la condición antes de ejecutar el bloque. Puede realizar cero iteraciones.

```
while (condicion) { /* instrucciones */ }
```

do-while

Ejecuta el bloque primero y después evalúa la condición, por lo que se ejecuta al menos una vez.

```
do { /* instrucciones */ } while (condicion);
```

Sumatorio

Una variable acumuladora permite sumar una cantidad indeterminada de valores. Es común utilizar un valor especial para indicar el final.

```
suma = 0; while (numero != 0) { suma += numero; scanf("%d", &numero); }
```

7. Procesamiento de datos en ficheros con bucles

Lecciones 21–23: cantidades, máximos, mínimos y precios.

Recorrido de archivos

Un ciclo permite procesar registros sucesivos hasta llegar al final del fichero. El patrón consiste en leer un dato, procesarlo y repetir.

Máximo y mínimo

Cuando se sabe que existe al menos un dato, una estrategia robusta es inicializar máximo y mínimo con el primer valor leído y después comparar los demás.

Negación lógica

El operador ! invierte una condición: una expresión verdadera pasa a ser falsa y una falsa pasa a ser verdadera.

8. Bucles for y aplicaciones básicas

Lecciones 24–26: for, cuadrados/cubos y números primos.

Estructura for

for reúne inicialización, condición e incremento en una sola estructura. Es apropiado cuando el recorrido está claramente definido.

```
for (int i = 0; i < 10; i++) { printf("%d\n", i); }
```

Cuadrados y cubos

Los ciclos pueden generar tablas de valores calculando $i*i$ e $i*i*i$ para cada número.

Números primos

Un número primo solo tiene como divisores positivos al 1 y a sí mismo. Para mejorar la eficiencia se pueden probar divisores hasta una cota relacionada con la raíz cuadrada del número.

9. Funciones en C y trazas de ejecución

Lecciones 27–29: funciones, trazas, MCD y MCM.

Funciones

Una función divide un problema en una parte reutilizable. Puede recibir parámetros y devolver un resultado.

```
int sumar(int a, int b) { return a + b; }
```

Media real

Para obtener una media real a partir de enteros se debe evitar la división entera.

```
media = (float)suma / cantidad;
```

Traza

Una traza sigue paso a paso la ejecución y los valores de las variables. Es útil para detectar errores lógicos.

MCD y MCM

El MCD puede obtenerse mediante el algoritmo de Euclides. Para valores no nulos, el MCM puede calcularse a partir del producto absoluto dividido entre el MCD.

10. Vectores numéricos

Lecciones 30–33: totales, medias, normalización, máximo, mínimo y búsqueda.

Arreglos o vectores

Un vector almacena varios elementos del mismo tipo. Los índices en C comienzan en cero.

```
int numeros[5] = {10, 20, 30, 40, 50};
```

Recorrido

Normalmente se utiliza un for para visitar cada posición.

```
for (int i = 0; i < 5; i++) { printf("%d\n", numeros[i]); }
```

Total y media

Se acumulan los elementos y se divide entre la cantidad de datos.

Normalización

Transforma datos a una escala común. Una técnica habitual utiliza el mínimo y máximo para obtener valores en un intervalo como [0,1].

Máximo y mínimo

Es conveniente inicializar ambos resultados con el primer elemento cuando el vector tiene al menos un dato.

Búsqueda secuencial

Revisa las posiciones una por una. Si no encuentra el elemento, puede devolver -1 cuando el resultado representa un índice.

11. Vectores dinámicos y cálculo de NIF

Lecciones 34–35: letra NIF y memoria dinámica.

Cálculo de NIF

La letra del DNI español se determina usando el residuo de dividir el número entre 23 y empleando ese resultado como índice en una cadena de letras.

Memoria dinámica

La memoria dinámica se reserva durante la ejecución. En C se pueden utilizar malloc/calloc y se libera con free().

```
int *v = malloc(n * sizeof(int));/* utilizar v */free(v);
```

12. Cadenas de caracteres y string.h

Lecciones 36–40: comparación, longitud, subcadenas y ficheros.

Cadenas

Una cadena es un arreglo de char terminado por el carácter nulo \0.

```
char nombre[] = "Ana";
```

Funciones string.h

strcmp() compara cadenas y strlen() obtiene su longitud. Estas funciones requieren incluir string.h.

```
#include <string.h>if (strcmp(a, b) == 0) { printf("Iguales");}
```

Subcadenas

Una subcadena es una porción de otra cadena. Es necesario validar los índices antes de copiar caracteres.

Ocurrencias en archivos

Las cadenas leídas de un fichero pueden analizarse para contar palabras u ocurrencias de un texto.

13. Matrices numéricas y de caracteres

Lecciones 41–43: recorrido, suma y matrices de caracteres.

Matrices

Una matriz bidimensional organiza datos mediante filas y columnas.

```
int matriz[3][3];
```

Recorrido

Se utilizan normalmente dos ciclos anidados: uno para filas y otro para columnas.

```
for (int i=0; i<3; i++) {   for (int j=0; j<3; j++) {       printf("%d ", matriz[i][j]);   }}
```

Suma de matrices

Dos matrices pueden sumarse si tienen las mismas dimensiones. Cada resultado se obtiene sumando elementos de igual posición.

```
C[i][j] = A[i][j] + B[i][j];
```

Matrices de caracteres

Pueden utilizarse para representar varias cadenas mediante filas de caracteres.

14. Estructuras y tipos definidos por el usuario

Lecciones 44–45: estructuras y typedef.

struct

Una estructura agrupa campos que pueden tener diferentes tipos, permitiendo representar entidades como alumnos.

```
struct Alumno {   char nombre[50];   int edad;   float promedio;};
```

Inicialización y acceso

Los campos se pueden inicializar al declarar la estructura y se acceden mediante el operador punto.

```
struct Alumno a = {"Ana", 19, 9.2};printf("%s", a.nombre);
```

typedef

typedef crea un nombre alternativo para un tipo y permite simplificar declaraciones de estructuras.

```
typedef struct { char nombre[50]; int edad;} Alumno;
```

15. Buenas prácticas de programación

- Usar nombres de variables claros y relacionados con su propósito.
- Inicializar variables antes de utilizarlas.
- Evitar acceder a posiciones fuera de los límites de un vector o matriz.
- Comprobar que fopen() haya abierto correctamente un fichero.
- Cerrar los ficheros con fclose().
- Liberar la memoria dinámica con free() cuando ya no sea necesaria.
- Dividir programas complejos en funciones pequeñas y reutilizables.
- Comprobar casos límite y entradas inesperadas.
- Utilizar comentarios cuando aporten información útil sobre la lógica.

16. Conclusión

El curso presenta una progresión lógica desde los elementos básicos de C hasta técnicas para organizar y procesar información. Primero se estudian variables, tipos, operadores y entrada/salida; después se incorporan condiciones, selección múltiple y ciclos; finalmente se trabajan funciones, archivos, vectores, memoria dinámica, cadenas, matrices y estructuras.

Estos conocimientos forman una base importante para continuar con programación estructurada y posteriormente con C++. La práctica mediante ejercicios es fundamental, porque permite transformar la sintaxis en habilidades para analizar problemas, diseñar algoritmos y construir programas funcionales.

Referencia

Cursa. (s. f.). Programación en Lenguaje C - Curso gratis.

<https://cursa.app/es/curso-gratis/programacion-en-lenguaje-c-defg>